Fila Estática

Professor: André Felipe Ribeiro Cordeiro
Disciplina: Estruturas de Dados

Departamento de Informática
Centro de Tecnologia
Universidade Estadual de Maringá

Introdução

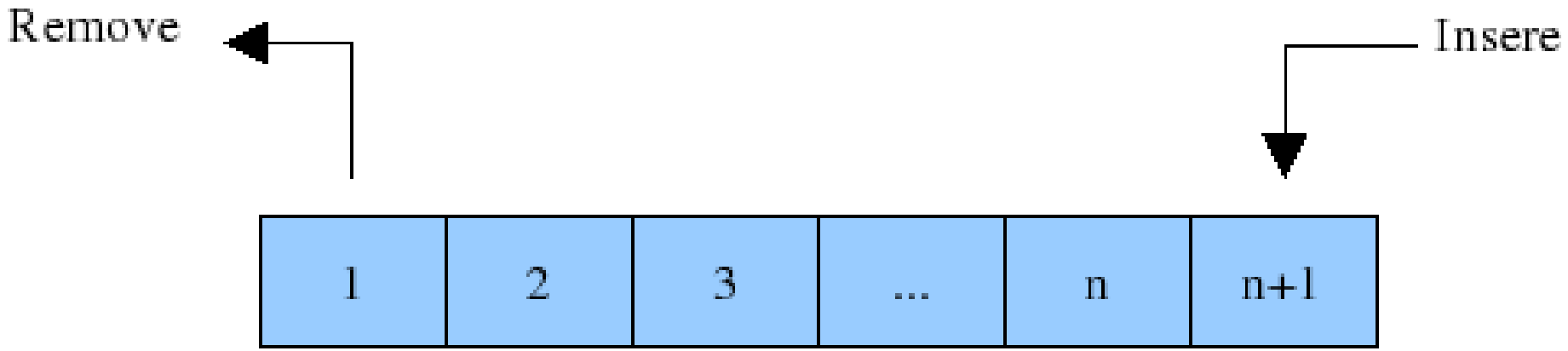
- Outra estrutura de dados bastante utilizada em computação é a **fila**.

- Na estrutura de fila, existem **regras** para o **acesso**, **inserção** e **remoção** de **itens**.

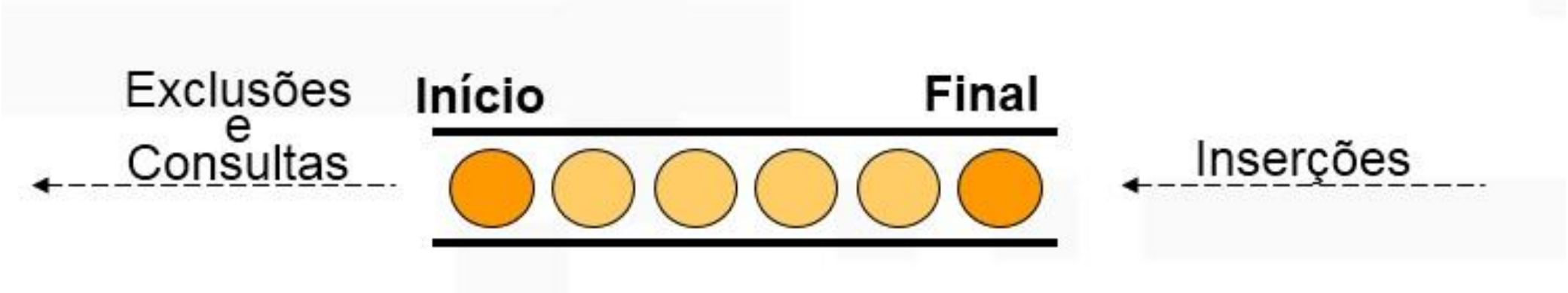
Regras

- **Acesso**
  - Dependente do TAD e/ou da aplicação.
- **Inserção**
  - Insere-se o item no **final da fila**.
- **Remoção**
  - Remove-se o item do **início da fila**.

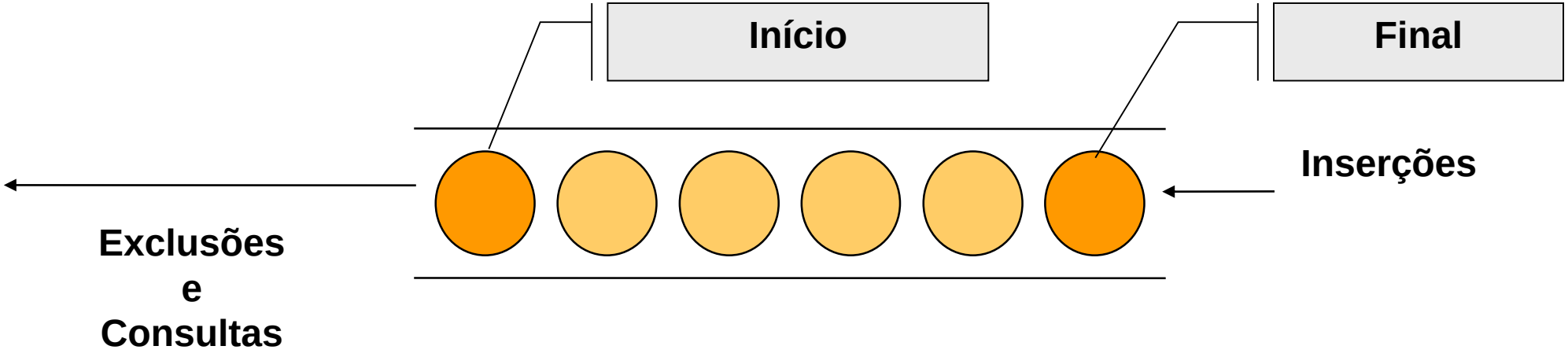
Representação



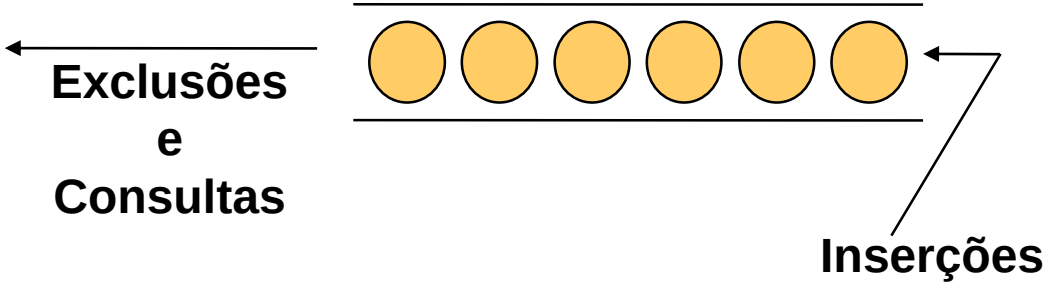
Representação

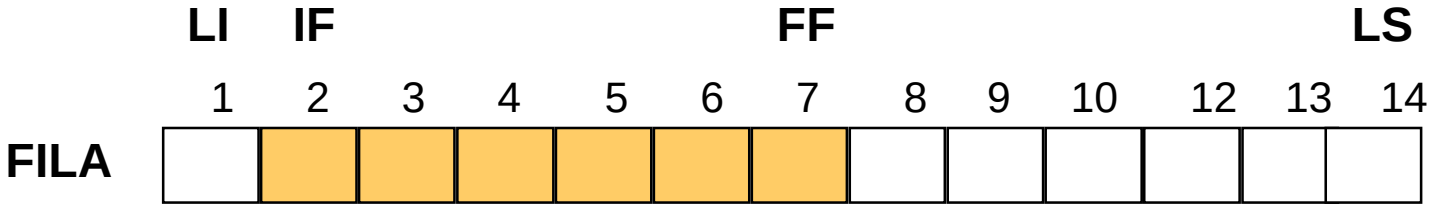


Representação



Simulação





LI : limite inferior da área

IF : início da fila

FF : final da fila

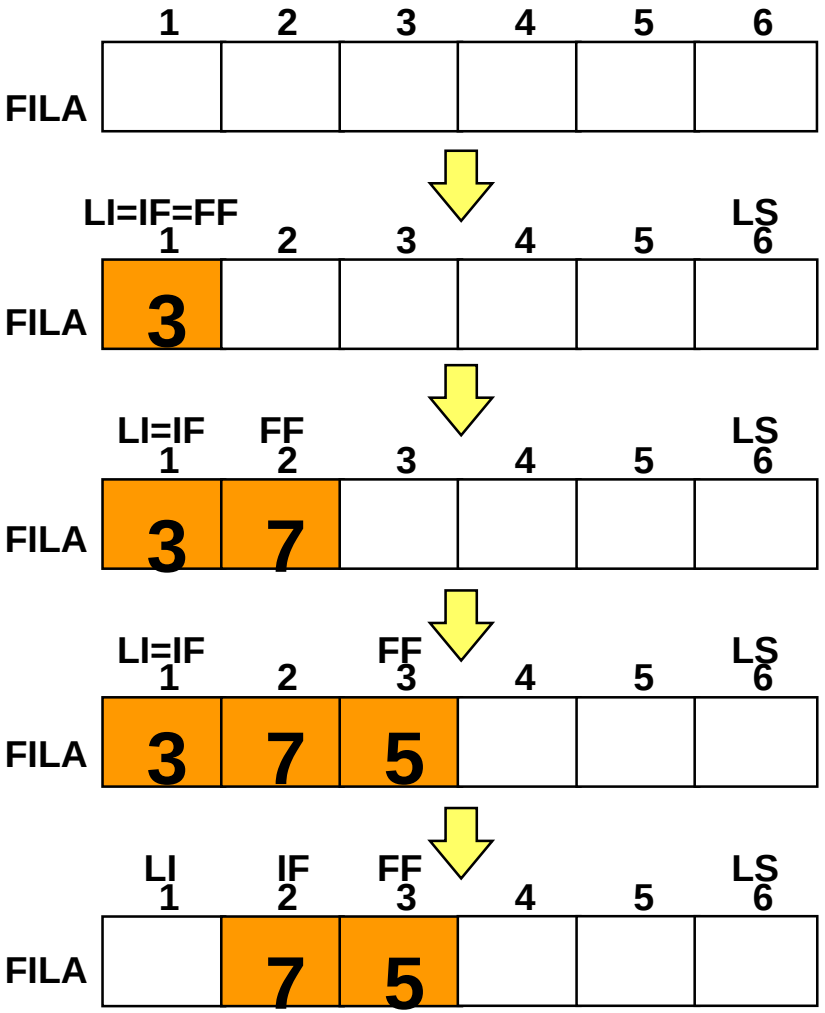
LS : limite superior da área

Fila vazia

IF = 0

Simulação

1. Inicializar a fila
2. Inserir um novo nodo com valor 3
3. Inserir um novo nodo com valor 7
4. Inserir um novo nodo com valor 5
5. Remover um nodo



Observações

- As **estruturas** de **pilha** e **fila** se **diferenciam** pela **ordem de saída** dos itens.

- Cada estrutura é organizada de uma **maneira específica**.

Observações

- **Pilha**
  - O último item que entra é o primeiro que sai.
  - **Last In**, **First Out** (LIFO)

- **Fila**
  - O primeiro item que entra é o primeiro que sai.
  - **First In**, **First Out** (FIFO)

Operações
Fila

- As seguintes operações podem ser utilizadas em diferentes aplicações:
  - **inicializar** a fila
  - **verificar** se a fila está **vazia**
  - **enfileirar item** na fila
  - **desenfileirar item** da fila
  - **imprimir** os **elementos** da fila

Implementação

```
#include <stdio.h>
#include <stdlib.h>
#define TAMANHO_MAXIMO  1000
```

```
typedef int TipoApontador;
typedef int TipoChave;

typedef struct {
  TipoChave chave;
  /* outros itens podem ser inseridos */
} TipoItem;
```

Implementação

```
typedef struct {
  TipoItem item[TAMANHO_MAXIMO];
  TipoApontador frente, tras;
} TipoFila;
```

```
void Inicializa(TipoFila *fila){
  fila->frente = 1;
  fila->tras = fila->frente;
}
```

Implementação

```
int Vazia(TipoFila fila){
  return (fila.frente == fila.tras);
}
```

```
void Imprime(TipoFila fila){
  int indice;

  for (indice = fila.frente;
                  indice < fila.tras; indice++)
    printf("%d\n", fila.item[indice].chave);
}
```

Implementação

```
void Enfileira(TipoItem elemento,
                                    TipoFila *fila){
  if (fila->tras % TAMANHO_MAXIMO + 1 ==
                                      fila->frente)
    printf(" Erro! Fila Cheia \n");
  else{
    fila->item[fila->tras] = elemento;
    fila->tras =
              fila->tras % TAMANHO_MAXIMO + 1;
  }
}
```

Implementação

```
void Desenfileira(TipoFila *fila,
                                TipoItem *elemento){
  if(Vazia(*fila))
    printf("Erro fila esta vazia\n");
  else{
    *elemento = fila->item[fila->frente];
    fila->frente =
            fila->frente % TAMANHO_MAXIMO + 1;
  }
}
```

Observações

- Outras operações poderiam ser consideradas, além das apresentadas.

- **Exemplos**
  - **Encontrar** o **maior elemento** da fila
  - **Contar** o **número de itens** da fila

Observações

- A estrutura de fila apresentada está organizada tal como outras filas “**não computacionais**”.

- Em um banco, a **primeira pessoa da fila** é a **primeira** a ser **atendida** (cenário esperado!).

Referências

- Introdução a Estruturas de Dados, Waldemar Celes, Renato Cerqueira e José Rangel, 2016.

- Projeto de Algoritmos com implementações em Pascal e C, Nivio Ziviani, 2011.

- Aulas Professor Rodolfo Miranda Pereira, UEM.